Bahria University, Islamabad

Department of Software Engineering

Artificial Intelligence Lab (Spring-2026)

Teacher: Engr. Saad Mazhar

Student : Talha Naveed

Enrollment : 01-131232-089

Lab Journal: 05

Date: 3rd March, 2026

Task No:	Task Wise Marks		Documentation Marks		Total Marks (20)
	Assigned	Obtained	Assigned	Obtained	
1	3		5		
2	3				
3	3				
4	3				
5	3				

Comments:

Signature

Lab No: 05 – A* Algorithm

Introduction

Introduction to A* algorithm in Python.

Tools Used

- Python
- VS Code

CODE: https://github.com/TALHA-089/AI-LAB-05.git

Task 1:

```
================================================================
Task 1: Graph Representation (Adjacency List)
================================================================

  S -> [B1(w=2), C(w=5), B2(w=4)]
  B1 -> [G(w=5)]
  B2 -> [C(w=1), F(w=1)]
  C -> [G(w=3)]
  G -> [C(w=2)]
  F -> [C(w=3)]
  E -> [B2(w=4)]
```

Task 2:

```
================================================================
Task 2: DFS (Depth-First Search)
================================================================
Path: S -> B2 -> F -> C -> G
Expanded nodes: 5

================================================================
Task 2: BFS (Breadth-First Search)
================================================================
Path: S -> B1 -> G
Expanded nodes: 5
```

Task 3:

```
============================================================
Task 2: DFS (Depth-First Search)
============================================================
Path: S -> B2 -> F -> C -> G
Expanded nodes: 5

============================================================
Task 2: BFS (Breadth-First Search)
============================================================
Path: S -> B1 -> G
Expanded nodes: 5
```

Task 4:

```
============================================================
Task 2: DFS (Depth-First Search)
============================================================
Path: S -> B2 -> F -> C -> G
Expanded nodes: 5

============================================================
Task 2: BFS (Breadth-First Search)
============================================================
Path: S -> B1 -> G
Expanded nodes: 5
```

Task 5:

```
============================================================
Task 5: Comparison: DFS vs BFS vs A*
============================================================

+-------------------+-------------------+-------------------+-------------------------+
| Criterion         | DFS               | BFS               | A*                      |
+-------------------+-------------------+-------------------+-------------------------+
| Data Structure    | Stack (LIFO)      | Queue (FIFO)      | Priority Queue (Heap)   |
+-------------------+-------------------+-------------------+-------------------------+
| Complete          | No (may loop      | Yes (for finite   | Yes (with admissible    |
|                   | without visited   | graphs)           | heuristic and finite    |
|                   | tracking)         |                   | branching factor)       |
+-------------------+-------------------+-------------------+-------------------------+
| Optimal           | No (finds any     | Yes (for uniform  | Yes (with admissible    |
|                   | path, not best)   | edge costs)       | heuristic)              |
+-------------------+-------------------+-------------------+-------------------------+
| Time Complexity   | O(b^m)            | O(b^d)            | O(b^d) worst case,      |
|                   | b = branching     | b = branching     | often much better       |
|                   | factor            | factor            | with a good heuristic   |
|                   | m = max depth     | d = solution      |                         |
|                   |                   | depth             |                         |
+-------------------+-------------------+-------------------+-------------------------+
| Space Complexity  | O(b*m)            | O(b^d)            | O(b^d)                  |
|                   | (linear in path   | (exponential,     | (stores all generated   |
|                   | length)           | stores all nodes  | nodes in open/closed    |
|                   |                   | at depth d)       | sets)                   |
+-------------------+-------------------+-------------------+-------------------------+
| When to Use       | Memory-limited    | Finding shortest  | Finding lowest-cost     |
|                   | scenarios, maze   | path in           | path in weighted        |
|                   | solving, topo-    | unweighted        | graphs such as GPS      |
|                   | logical sorting,  | graphs, level-    | navigation, robotics    |
|                   | cycle detection   | order traversal,  | path planning, and      |
|                   | in graphs         | peer-to-peer      | game AI where a good    |
|                   |                   | networking        | heuristic is available  |
+-------------------+-------------------+-------------------+-------------------------+

Heuristic Justification:
h(n) = 0 is used for all nodes, which is trivially admissible because it
never overestimates the actual cost to reach the goal (the actual cost is
always >= 0). With h(n) = 0, A* reduces to Dijkstra's algorithm, which
guarantees finding the optimal (minimum-cost) path. This choice is
appropriate when no domain-specific distance estimate is available for
the graph nodes.
```